```
import re

# --- Data Structures for Logic Terms ---
class Term:
    pass

class Constant(Term):
    def __init__(self, name):
        self.name = name
    def __repr__(self):
        return self.name

class Variable(Term):
    def __init__(self, name):
        self.name = name
    def __repr__(self):
        return self.name

class Predicate(Term):
    def __init__(self, name, args):
        self.name = name
        self.args = args
    def __repr__(self):
        return f"{self.name}({', '.join(map(str, self.args))})"

# --- Helper to follow the substitution chain during lookup ---
def resolve(term, subst):
    while isinstance(term, Variable) and term.name in subst:
        term = subst[term.name]
    if isinstance(term, Predicate):
        return Predicate(term.name, [resolve(arg, subst) for arg in term.args])
    return term

# --- Unification Engine ---
def unify(x, y, subst=None):
    if subst is None:
        subst = {}
    if subst == "FAIL":
        return "FAIL"

    # Resolve terms using current bindings before comparing
    x = resolve(x, subst)
    y = resolve(y, subst)

    if str(x) == str(y):
        return subst
    if isinstance(x, Variable):
        return unify_var(x, y, subst)
    if isinstance(y, Variable):
        return unify_var(y, x, subst)
    if isinstance(x, Predicate) and isinstance(y, Predicate):
        if x.name != y.name or len(x.args) != len(y.args):
            return "FAIL"
        for arg1, arg2 in zip(x.args, y.args):
            subst = unify(arg1, arg2, subst)
            if subst == "FAIL":
                return "FAIL"
        return subst
    return "FAIL"

def unify_var(var, x, subst):
    if var.name == getattr(x, 'name', None):
        return subst
    if occurs_check(var, x, subst):
        return "FAIL"

    # Store the direct match without modifying existing keys backwards
    subst[var.name] = x
    return subst

def occurs_check(var, x, subst):
    x = resolve(x, subst)
    if var.name == getattr(x, 'name', None):
        return True
    if isinstance(x, Predicate):
```

```
        return any(occurs_check(var, arg, subst) for arg in x.args)
    return False

# --- Quick String Parser for User Input ---
def tokenize(s):
    return re.findall(r'[A-Za-z0-9]+|[\(\),]', s.replace(" ", ""))

def parse_term(tokens, index):
    token = tokens[index]
    index += 1

    if index < len(tokens) and tokens[index] == '(':
        name = token
        index += 1
        args = []
        while tokens[index] != ')':
            arg, index = parse_term(tokens, index)
            args.append(arg)
            if tokens[index] == ',':
                index += 1
        index += 1
        return Predicate(name, args), index
    else:
        if token.lower() in ['x', 'y', 'z', 'u', 'v', 'w']:
            return Variable(token), index
        else:
            return Constant(token), index

def parse_expression(s):
    try:
        tokens = tokenize(s)
        expr, _ = parse_term(tokens, 0)
        return expr
    except Exception:
        return None

# --- Main Runtime Loop ---
if __name__ == "__main__":
    print("=== Updated FOL Unification Engine ===")
    print("Variables: x, y, z, u, v, w\n")

    while True:
        raw_expr1 = input("Enter Expression 1: ").strip()
        if raw_expr1.lower() == 'exit': break

        raw_expr2 = input("Enter Expression 2: ").strip()
        if raw_expr2.lower() == 'exit': break

        expr1 = parse_expression(raw_expr1)
        expr2 = parse_expression(raw_expr2)

        if not expr1 or not expr2:
            print("❌ Invalid syntax. Try again.\n")
            continue

        result = unify(expr1, expr2)

        if result == "FAIL":
            print("🔴 Unification Result: FAIL\n")
        else:
            print("🟢 Unification Result: SUCCESS (MGU)")
            print(f"   Substitution: {result}\n")
        print("-" * 50)
```

```
=== Updated FOL Unification Engine ===
Variables: x, y, z, u, v, w

Enter Expression 1: p(b,x,f(g(z)))
Enter Expression 2: p(z,f(y),f(y))
🟢 Unification Result: SUCCESS (MGU)
   Substitution: {'z': b, 'x': f(y), 'y': g(b)}

--------------------------------------------------
Enter Expression 1: P(f(x),A)
Enter Expression 2: P(B,y)
🔴 Unification Result: FAIL
```

```
--------------------------------------------------
Enter Expression 1:
```